Rafał Szczerba – STEM agent

Design

I decided that my solution will consist of 4 crucial elements:

1. **Discovery Agent** – this is initial agent, that processes part of dataset to determine general task description and evaluation rules. In case of my project I have set this part to be about 20% of whole original dataset.
   Why not pass entire dataset? To avoid overfitting data. There were iterations, where this agent had hardcoded all of data from dataset, what resulted in fake 100% coverage.
2. **STEM Agent** – this agent is responsible for creating "blueprint" of target worker agent. It's task is to determine features of worker, like system prompt, temperature, tools to use and reasoning techniques. In final edition of the project I implemented only part of those features (more details below).
3. **Worker** – this is target agent. It's sole purpose is to perform task given by STEM agent and return predictable, correct answer. It does not have hard-coded scheme, temperature nor system prompt.
4. **Judge** – it's task is to determine if solution generated by worker is correct or not.
   If not, then it must return feedback to STEM Agent, that will be used in next iteration.
   There are 2 main ways to implement Judge:
   - Deterministic Judge – handwritten tests, that will return a boolean with correctness of data, as well as expected output and actual output
     Benefits:
     - Predictable behaviour
     - Easy to implement
     - Does not use extra tokens

     Drawbacks:
     - Not flexible
     - Require known structure of request/response
   - LLM as a Judge – agent, that will check correctness of output
     Benefits:
     - Flexible
     - Does not require known structure (if connected to Discovery agent, otherwise it's required to specify text prompt)
     - Returns tailored feedback

     Drawbacks:
     - Uses tokens
     - Can hallucinate

I decided to start with deterministic Judge and later move to LLM as a Judge.

Domain and scope

The agent takes unstructured data and an expected response. Based on that, it deduces what is its goal and how to achieve it.

The scope is confined to an evolutionary loop. Due to complexity of determining threshold for learning, I decided to set evolution for constant number of iterations with backtracking to prevent degradation of next agents.

It uses 3 models: one heavy for STEM agent, heavy/medium for Judge and basic for Worker.

What surprised me?/Interesting observations

1. When I was thinking about false responses it was very easy to fall into thinking, that we need another agent to make sure, that the former one is not doing something against its designed behaviour. I came to the conclusion that at some point human supervision might be beneficial.
2. Models are very good at finding flaws of the Judge. In one iteration all of the tests passed. I was surprised, because it has never happened before. It turned out, that STEM agent found a way to bypass rules of the Judge by giving specific prompts.
3. Selection of used models is crucial. I reached the following conclusions:
    a. Too weak STEM model -> no improvement in learning
    b. Too weak Judge models -> false positives/false negatives very often
    c. Too weak worker -> it's difficult to maintain scheme and get reliable response
    d. Too strong STEM in comparison to Judge -> Judge can be "tricked"
4. Temperature is extremely important:
    a. Judge has to be at 0 to prevent hallucinations
    b. Discovery works best at 0,3-0,5
    c. STEM agent works best at 0,5-0,7
    d. Worker temperature set by STEM agent usually settles at about 0,4-0,6
5. Even small changes in system prompt may result in completely different results. Adding one rule can decrease or increase accuracy of the agent by huge percentage (in my case maximal change was 20%).

Problems/What failed

During implementation of the agent, I encountered multiple issues. Some of them were trivial and easy to fix, others were more complex.

1. **Context Loss:** After implementing more rules to STEM Agent, it became significantly worse at its task. It generated more detailed prompts for workers, which resulted in losing context and “forgetting” about different aspects of the prompt.
   I discovered that the best way to ensure best results is to keep prompts concise, short and simple.
2. **Infinite Feedback Noise**: The STEM agent is fed a history list containing every previous iteration's prompt and all of the Judge's error logs. As the iterations progressed, the context window filled up with outdated errors, causing the STEM agent to try and fix edge-cases that it had already resolved in previous generations.
   I didn’t have enough time to fix it, but probably I would solve it by maintaining map of records with issues and cleaning it when next test passes.
3. **JSON Schema Enforcement:** Because the worker agent is a relatively small model, it frequently failed to return pure JSON (e.g., adding markdown code blocks like ```json). If the STEM agent did not explicitly command the worker to avoid markdown, the LLM Judge would sometimes reject the output due to strict string matching failures.

What would I do with more time?

1. I would implement tools for searching web, validating URLs. I would also reconsider adding interpreter for generating needed tools by model itself, but this is difficult due to potential security issues.
2. I would solve infinite feedback noise issue by maintaining map of records with issues and cleaning it when next test passes.
3. I would add reasoning techniques like CoT to choose by model. I actually tried it, but it resulted in a lot of redundant text in prompt and basic models were easily confused by this. It is on branch “add-reasoning-tools” on GIT.

Results:

Dataset: Emails/Tickets from customers

Response: {department, priority}

ITERATION 1:

- Temperature 0.5
- Success Rate 71%

ITERATION 2:

- Temperature 0.5
- Success Rate 86%

ITERATION 3:

- Temperature 0.4
- Success Rate 76%

ITERATION 4:

- Temperature 0.7
- Success Rate 86%

ITERATION 5:

- Temperature 0.5
- Success Rate 95%

Our observed increase in fifth iteration equals about 24% in comparison to first iteration.

We can also observe fluctuations in temperature. System prompt is about twice the size of the original one.